CHAPTER 8

Searching and Sorting Arrays

TOPICS

8.1 Focus on Software Engineering: Introduction to Search Algorithms
8.2 Focus on Problem Solving and Program Design: A Case Study
8.3 Focus on Software Engineering: Introduction to Sorting Algorithms
8.4 Focus on Problem Solving and Program Design: A Case Study
8.5 Sorting and Searching `vectors`

8.1 Focus on Software Engineering: Introduction to Search Algorithms

CONCEPT: **A search algorithm is a method of locating a specific item in a larger collection of data. This section discusses two algorithms for searching the contents of an array.**

It's very common for programs not only to store and process data stored in arrays, but also to search arrays for specific items. This section will show you two methods of searching an array: the linear search and the binary search. Each has its advantages and disadvantages.

The Linear Search

The *linear search* is a very simple algorithm. Also known as the *sequential search* algorithm, it uses a loop to sequentially step through an array, starting with the first element. It compares each element with the value being searched for, and stops when either the value is found or the end of the array is encountered. If the value being searched for is not in the array, the algorithm will unsuccessfully search to the end of the array.

Here is the pseudocode for a function that performs the linear search:

```
Set found to false
Set position to -1
Set index to 0
```

```
While found is false and index < number of elements
   If list[index] is equal to search value
      found = true
      position = index
   End If
   Add 1 to index
End While
Return position
```

The `linearSearch` function shown below is an example of C++ code used to perform a linear search on an integer array. The array `arr`, which has a maximum of `size` elements, is searched for an occurrence of the number stored in `value`. If the number is found, its array subscript is returned. Otherwise, –1 is returned, indicating the value did not appear in the array.

```
int linearSearch (const int arr[], int size, int value)
{
  int index = 0;        // Used as a subscript to search the array
  int position = -1;    // To record the position of the search value
  bool found = false;   // Flag to indicate if the value was found

  while (index < size && !found)
  {
     if (arr[index] == value)       // If the value is found
     {
        found = true;               // Set the flag
        position = index;           // Record the value's subscript
     }
     index++;                       // Go to the next element
  }
  return position;                  // Return the position, or -1
}
```



NOTE: The reason –1 is returned when the search value is not found in the array is because –1 is not a valid subscript.

Program 8-1 is a complete program that uses the `linearSearch` function. It searches the five-element array `tests` to find a score of 100.

Program 8-1

```
1  // This program demonstrates the linear search algorithm.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototype
6  int linearSearch(const int[], int, int);
7
8  int main()
9  {
10    const int SIZE = 5;
11    int tests[SIZE] = { 87, 75, 98, 100, 82 };
12    int results;
```

```
13
14      // Search the array for 100.
15      results = linearSearch(tests, SIZE, 100);
16
17      // If linearSearch returned -1, then 100 was not found.
18      if (results == -1)
19         cout << "You did not earn 100 points on any test\n";
20      else
21      {
22         // Otherwise results contains the subscript of
23         // the first 100 in the array.
24         cout << "You earned 100 points on test ";
25         cout << (results + 1) << endl;
26      }
27      return 0;
28   }
29
30   //*****************************************************************
31   // The linearSearch function performs a linear search on an       *
32   // integer array. The array arr, which has a maximum of size      *
33   // elements, is searched for the number stored in value. If the   *
34   // number is found, its array subscript is returned. Otherwise,   *
35   // -1 is returned indicating the value was not in the array.      *
36   //*****************************************************************
37   int linearSearch(const int arr[], int size, int value)
38   {
39      int index = 0;         // Used as a subscript to search array
40      int position = -1;     // To record position of search value
41      bool found = false;    // Flag to indicate if the value was found
42
43      while (index < size && !found)
44      {
45         if (arr[index] == value)     // If the value is found
46         {
47            found = true;             // Set the flag
48            position = index;         // Record the value's subscript
49         }
50         index++;                     // Go to the next element
51      }
52      return position;                // Return the position, or -1
53   }
```

Program Output

```
You earned 100 points on test 4
```

Inefficiency of the Linear Search

The advantage of the linear search is its simplicity. It is very easy to understand and implement. Furthermore, it doesn't require the data in the array to be stored in any particular order. Its disadvantage, however, is its inefficiency. If the array being searched contains 20,000 elements, the algorithm will have to look at all 20,000 elements in order to find

a value stored in the last element (so the algorithm actually reads an element of the array 20,000 times).

In an average case, an item is just as likely to be found near the beginning of the array as near the end. Typically, for an array of N items, the linear search will locate an item in N/2 attempts. If an array has 50,000 elements, the linear search will make a comparison with 25,000 of them in a typical case. This is assuming, of course, that the search item is consistently found in the array. (N/2 is the average number of comparisons. The maximum number of comparisons is always N.)

When the linear search fails to locate an item, it must make a comparison with every element in the array. As the number of failed search attempts increases, so does the average number of comparisons. Obviously, the linear search should not be used on large arrays if the speed is important.

The Binary Search



VideoNote
The Binary Search

The *binary search* is a clever algorithm that is much more efficient than the linear search. Its only requirement is that the values in the array be sorted in order. Instead of testing the array's first element, this algorithm starts with the element in the middle. If that element happens to contain the desired value, then the search is over. Otherwise, the value in the middle element is either greater than or less than the value being searched for. If it is greater, then the desired value (if it is in the list) will be found somewhere in the first half of the array. If it is less, then the desired value (again, if it is in the list) will be found somewhere in the last half of the array. In either case, half of the array's elements have been eliminated from further searching.

If the desired value wasn't found in the middle element, the procedure is repeated for the half of the array that potentially contains the value. For instance, if the last half of the array is to be searched, the algorithm immediately tests *its* middle element. If the desired value isn't found there, the search is narrowed to the quarter of the array that resides before or after that element. This process continues until either the value being searched for is found, or there are no more elements to test.

Here is the pseudocode for a function that performs a binary search on an array:

```
Set first to 0
Set last to the last subscript in the array
Set found to false
Set position to -1
While found is not true and first is less than or equal to last
   Set middle to the subscript halfway between array[first]
   and array[last]
   If array[middle] equals the desired value
      Set found to true
      Set position to middle
   Else If array[middle] is greater than the desired value
      Set last to middle - 1
   Else
      Set first to middle + 1
   End If
End While
Return position
```

This algorithm uses three index variables: `first`, `last`, and `middle`. The `first` and `last` variables mark the boundaries of the portion of the array currently being searched. They are initialized with the subscripts of the array's first and last elements. The subscript of the element halfway between `first` and `last` is calculated and stored in the `middle` variable. If the element in the middle of the array does not contain the search value, the `first` or `last` variables are adjusted so only the top or bottom half of the array is searched during the next iteration. This cuts the portion of the array being searched in half each time the loop fails to locate the search value.

The function `binarySearch` shown in the following example is used to perform a binary search on an integer array. The first parameter, `array`, which has a maximum of `numElems` elements, is searched for an occurrence of the number stored in `value`. If the number is found, its array subscript is returned. Otherwise, –1 is returned indicating the value did not appear in the array.

```
int binarySearch(const int array[], int numElems, int value)
{
   int first = 0,                        // First array element
       last = numElems - 1,              // Last array element
       middle,                           // Midpoint of search
       position = -1;                    // Position of search value
   bool found = false;                   // Flag

   while (!found && first <= last)
   {
       middle = (first + last) / 2;      // Calculate midpoint
       if (array[middle] == value)       // If value is found at mid
       {
           found = true;
           position = middle;
       }
       else if (array[middle] > value)   // If value is in lower half
           last = middle - 1;
       else
           first = middle + 1;           // If value is in upper half
   }
   return position;
}
```

Program 8-2 is a complete program using the `binarySearch` function. It searches an array of employee ID numbers for a specific value.

Program 8-2

```
1  // This program demonstrates the binarySearch function, which
2  // performs a binary search on an integer array.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototype
7  int binarySearch(const int [], int, int);
8  const int SIZE = 20;
```

(program continues)

Program 8-2 *(continued)*

```
 9
10  int main()
11  {
12      // Array with employee IDs sorted in ascending order.
13      int idNums[SIZE] = {101, 142, 147, 189, 199, 207, 222,
14                          234, 289, 296, 310, 319, 388, 394,
15                          417, 429, 447, 521, 536, 600};
16      int results;   // To hold the search results
17      int empID;     // To hold an employee ID
18
19      // Get an employee ID to search for.
20      cout << "Enter the employee ID you wish to search for: ";
21      cin >> empID;
22
23      // Search for the ID.
24      results = binarySearch(idNums, SIZE, empID);
25
26      // If results contains -1 the ID was not found.
27      if (results == -1)
28         cout << "That number does not exist in the array. \n";
29      else
30      {
31            // Otherwise results contains the subscript of
32            // the specified employee ID in the array.
33            cout << "That ID is found at element " << results;
34            cout << " in the array.\n";
35      }
36      return 0;
37  }
38
39  //*************************************************************
40  // The binarySearch function performs a binary search on an     *
41  // integer array. array, which has a maximum of size elements,  *
42  // is searched for the number stored in value. If the number is *
43  // found, its array subscript is returned. Otherwise, -1 is     *
44  // returned indicating the value was not in the array.          *
45  //*************************************************************
46
47  int binarySearch(const int array[], int size, int value)
48  {
49        int first = 0,          // First array element
50            last = size - 1,    // Last array element
51            middle,             // Midpoint of search
52            position = -1;      // Position of search value
53        bool found = false;     // Flag
54
55      while (!found && first <= last)
56      {
57          middle = (first + last) / 2;      // Calculate midpoint
58          if (array[middle] == value)       // If value is found at mid
```

```
59          {
60               found = true;
61               position = middle;
62          }
63          else if (array[middle] > value)// If value is in lower half
64               last = middle - 1;
65          else
66               first = middle + 1; // If value is in upper half
67      }
68      return position;
69  }
```

Program Output with Example Input Shown in Bold

```
Enter the employee ID you wish to search for: 199 Enter
That ID is found at element 4 in the array.
```



WARNING! Notice the array in Program 8-2 is initialized with its values already sorted in ascending order. The binary search algorithm will not work properly unless the values in the array are sorted.

The Efficiency of the Binary Search

Obviously, the binary search is much more efficient than the linear search. Every time it makes a comparison and fails to find the desired item, it eliminates half of the remaining portion of the array that must be searched. For example, consider an array with 1,000 elements. If the binary search fails to find an item on the first attempt, the number of elements that remains to be searched is 500. If the item is not found on the second attempt, the number of elements that remains to be searched is 250. This process continues until the binary search has either located the desired item or determined that it is not in the array. With 1,000 elements, this takes no more than 10 comparisons. (Compare this to the linear search, which would make an average of 500 comparisons!)

Powers of 2 are used to calculate the maximum number of comparisons the binary search will make on an array of any size. (A power of 2 is 2 raised to the power of some number.) Simply find the smallest power of 2 that is greater than or equal to the number of elements in the array. For example, a maximum of 16 comparisons will be made on an array of 50,000 elements (2^{16} = 65,536), and a maximum of 20 comparisons will be made on an array of 1,000,000 elements (2^{20} = 1,048,576).

8.2 Focus on Problem Solving and Program Design: A Case Study

The Demetris Leadership Center (DLC, Inc.) publishes the books, DVDs, and audio CDs listed in Table 8-1.

Table 8-1 Products

Product Title	Product Description	Product Number	Unit Price
Six Steps to Leadership	Book	914	$12.95
Six Steps to Leadership	Audio CD	915	$14.95
The Road to Excellence	DVD	916	$18.95
Seven Lessons of Quality	Book	917	$16.95
Seven Lessons of Quality	Audio CD	918	$21.95
Seven Lessons of Quality	DVD	919	$31.95
Teams Are Made, Not Born	Book	920	$14.95
Leadership for the Future	Book	921	$14.95
Leadership for the Future	Audio CD	922	$16.95

The manager of the Telemarketing Group has asked you to write a program that will help order-entry operators look up product prices. The program should prompt the user to enter a product number, and will then display the title, description, and price of the product.

Variables

Table 8-2 lists the variables needed:

Table 8-2 Variables

Variable	Description
`NUM_PRODS`	A constant integer initialized with the number of products the Demetris Leadership Center sells. This value will be used in the definition of the program's array.
`MIN_PRODNUM`	A constant integer initialized with the lowest product number.
`MAX_PRODNUM`	A constant integer initialized with the highest product number.
`id`	Array of integers. Holds each product's number.
`title`	Array of strings, initialized with the titles of products.
`description`	Array of strings, initialized with the descriptions of each product.
`prices`	Array of `doubles`. Holds each product's price.

Modules

The program will consist of the functions listed in Table 8-3.

Table 8-3 Functions

Function	Description
`main`	The program's `main` function. It calls the program's other functions.
`getProdNum`	Prompts the user to enter a product number. The function validates input and rejects any value outside the range of correct product numbers.
`binarySearch`	A standard binary search routine. Searches an array for a specified value. If the value is found, its subscript is returned. If the value is not found, –1 is returned.
`displayProd`	Uses a common subscript into the `title`, `description`, and `prices` arrays to display the title, description, and price of a product.

Function main

Function main contains the variable definitions and calls the other functions. Here is its pseudocode:

```
do
   Call getProdNum
   Call binarySearch
   If binarySearch returned -1
      Inform the user that the product number was not found
   else
      Call displayProd
   End If
   Ask the user if the program should repeat
While the user wants to repeat the program
```

Here is its actual C++ code:

```
do
{
   // Get the desired product number.
   prodNum = getProdNum();

   // Search for the product number.
   index = binarySearch(id, NUM_PRODS, prodNum);

   // Display the results of the search.
   if (index == -1)
      cout << "That product number was not found.\n";
   else
      displayProd(title, description, prices, index);

   // Does the user want to do this again?
   cout << "Would you like to look up another product? (y/n) ";
   cin >> again;
} while (again == 'y' || again == 'Y');
```

The named constant NUM_PRODS is defined globally and initialized with the value 9. The arrays id, title, description, and prices will already be initialized with data.

The getProdNum Function

The getProdNum function prompts the user to enter a product number. It tests the value to ensure it is in the range of 914–922 (which are the valid product numbers). If an invalid value is entered, it is rejected and the user is prompted again. When a valid product number is entered, the function returns it. The pseudocode is shown below.

```
Display a prompt to enter a product number
Read prodNum
While prodNum is invalid
   Display an error message
   Read prodNum
End While
Return prodNum
```

Here is the actual C++ code:

```
int getProdNum()
{
   int prodNum;

   cout << "Enter the item's product number: ";
   cin >> prodNum;
   // Validate input.
   while (prodNum < MIN_PRODNUM || prodNum > MAX_PRODNUM)
   {
      cout << "Enter a number in the range of " << MIN_PRODNUM;
      cout <<" through " << MAX_PRODNUM << ".\n";
      cin >> prodNum;
   }
   return prodNum;
}
```

The `binarySearch` Function

The `binarySearch` function is identical to the function discussed earlier in this chapter.

The `displayProd` Function

The `displayProd` function has parameter variables named `title`, `desc`, `price`, and `index`. These accept as arguments (respectively) the `title`, `description`, and `price` arrays, and a subscript value. The function displays the data stored in each array at the subscript passed into `index`. Here is the C++ code:

```
void displayProd(const string title[], const string desc[],
                 const double price[], int index)
{
   cout << "Title: " << title[index] << endl;
   cout << "Description: " << desc[index] << endl;
   cout << "Price: $" << price[index] << endl;
}
```

The Entire Program

Program 8-3 shows the entire program's source code.

Program 8-3

```
1  // Demetris Leadership Center (DLC) product lookup program
2  // This program allows the user to enter a product number
3  // and then displays the title, description, and price of
4  // that product.
5  #include <iostream>
6  #include <string>
7  using namespace std;
8
```

```
const int NUM_PRODS = 9;       // The number of products produced
const int MIN_PRODNUM = 914;   // The lowest product number
const int MAX_PRODNUM = 922;   // The highest product number

// Function prototypes
int getProdNum();
int binarySearch(const int [], int, int);
void displayProd(const string [], const string [], const double [], int);

int main()
{
    // Array of product IDs
    int id[NUM_PRODS] = {914, 915, 916, 917, 918, 919, 920,
                         921, 922};

    // Array of product titles
    string title[NUM_PRODS] =
        { "Six Steps to Leadership",
          "Six Steps to Leadership",
          "The Road to Excellence",
          "Seven Lessons of Quality",
          "Seven Lessons of Quality",
          "Seven Lessons of Quality",
          "Teams Are Made, Not Born",
          "Leadership for the Future",
          "Leadership for the Future"
        };

    // Array of product descriptions
    string description[NUM_PRODS] =
        { "Book", "Audio CD", "DVD",
          "Book", "Audio CD", "DVD",
          "Book", "Book", "Audio CD"
        };

    // Array of product prices
    double prices[NUM_PRODS] = {12.95, 14.95, 18.95, 16.95, 21.95,
                                31.95, 14.95, 14.95, 16.95};

    int prodNum; // To hold a product number
    int index;   // To hold search results
    char again;  // To hold a Y or N answer

    do
    {
        // Get the desired product number.
        prodNum = getProdNum();

        // Search for the product number.
        index = binarySearch(id, NUM_PRODS, prodNum);

        // Display the results of the search.
        if (index == -1)
```

(program continues)

Program 8-3 *(continued)*

```
62              cout << "That product number was not found.\n";
63          else
64              displayProd(title, description, prices, index);
65
66          // Does the user want to do this again?
67          cout << "Would you like to look up another product? (y/n) ";
68          cin >> again;
69      } while (again == 'y' || again == 'Y');
70      return 0;
71  }
72
73  //*****************************************************
74  // Definition of getProdNum function                  *
75  // The getProdNum function asks the user to enter a *
76  // product number. The input is validated, and when *
77  // a valid number is entered, it is returned.       *
78  //*****************************************************
79
80  int getProdNum()
81  {
82      int prodNum; // Product number
83
84      cout << "Enter the item's product number: ";
85      cin >> prodNum;
86      // Validate input
87      while (prodNum < MIN_PRODNUM || prodNum > MAX_PRODNUM)
88      {
89          cout << "Enter a number in the range of " << MIN_PRODNUM;
90          cout <<" through " << MAX_PRODNUM << ".\n";
91          cin >> prodNum;
92      }
93      return prodNum;
94  }
95
96  //*********************************************************************
97  // Definition of binarySearch function                                *
98  // The binarySearch function performs a binary search on an      *
99  // integer array. array, which has a maximum of numElems         *
100 // elements, is searched for the number stored in value. If the  *
101 // number is found, its array subscript is returned. Otherwise,  *
102 // -1 is returned indicating the value was not in the array.     *
103 //*********************************************************************
104
105 int binarySearch(const int array[], int numElems, int value)
106 {
107     int first = 0,              // First array element
108         last = numElems - 1,    // Last array element
109         middle,                 // Midpoint of search
110         position = -1;          // Position of search value
111     bool found = false;         // Flag
```

```
112
113       while (!found && first <= last)
114       {
115            middle = (first + last) / 2; // Calculate midpoint
116            if (array[middle] == value)  // If value is found at mid
117            {
118                 found = true;
119                 position = middle;
120            }
121            else if (array[middle] > value) // If value is in lower half
122                 last = middle - 1;
123            else
124                 first = middle + 1;         // If value is in upper half
125       }
126       return position;
127  }
128
129  //***************************************************************
130  // The displayProd function accepts three arrays and an int.    *
131  // The arrays parameters are expected to hold the title,        *
132  // description, and prices arrays defined in main. The index    *
133  // parameter holds a subscript. This function displays the      *
134  // information in each array contained at the subscript.        *
135  //***************************************************************
136
137  void displayProd(const string title[], const string desc[],
138                   const double price[], int index)
139  {
140       cout << "Title: " << title[index] << endl;
141       cout << "Description: " << desc[index] << endl;
142       cout << "Price: $" << price[index] << endl;
143  }
```

Program Output with Example Input Shown in Bold

```
Enter the item's product number: 916 Enter
Title: The Road to Excellence
Description: DVD
Price: $18.95
Would you like to look up another product? (y/n) y Enter
Enter the item's product number: 920 Enter
Title: Teams Are Made, Not Born
Description: Book
Price: $14.95
Would you like to look up another product? (y/n) n Enter
```



Checkpoint

8.1 Describe the difference between the linear search and the binary search.

8.2 On average, with an array of 20,000 elements, how many comparisons will the linear search perform? (Assume the items being searched for are consistently found in the array.)

8.3 With an array of 20,000 elements, what is the maximum number of comparisons the binary search will perform?

8.4 If a linear search is performed on an array, and it is known that some items are searched for more frequently than others, how can the contents of the array be reordered to improve the average performance of the search?

8.3 Focus on Software Engineering: Introduction to Sorting Algorithms

CONCEPT: Sorting algorithms are used to arrange data into some order.

Sorting Algorithms

Many programming tasks require the data in an array be sorted in some order. Customer lists, for instance, are commonly sorted in alphabetical order. Student grades might be sorted from highest to lowest, and product codes could be sorted so that all the products of the same color are stored together. To sort the data in an array, the programmer must use an appropriate sorting algorithm. A *sorting algorithm* is a technique for stepping through an array and rearranging its contents in some order.

The data in an array can be sorted in either ascending or descending order. If an array is sorted in *ascending order,* it means the values in the array are stored from lowest to highest. If the values are sorted in *descending order*, they are stored from highest to lowest. This section will introduce two simple sorting algorithms: the *bubble sort* and the *selection sort*.

The Bubble Sort

The bubble sort is an easy way to arrange data in ascending or descending order. It is called the *bubble sort* algorithm because as it makes passes through and compares the elements of the array, certain values "bubble" toward the end of the array with each pass. For example, if you are using the algorithm to sort an array in ascending order, the larger values move toward the end. If you are using the algorithm to sort an array in descending order, the smaller values move toward the end. In this section, you will see how the bubble sort algorithm can be used to sort an array in ascending order.

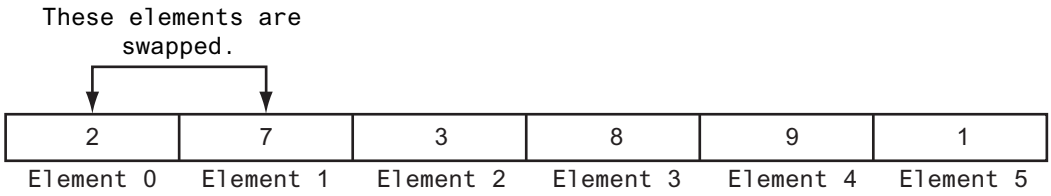
Suppose we have the array shown in Figure 8-1. Let's see how the bubble sort can be used in arranging the array's elements in ascending order.

Figure 8-1 An array



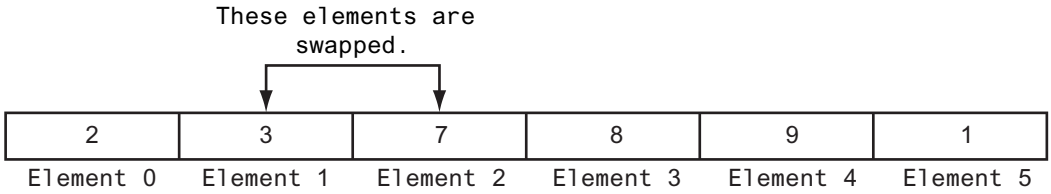
The bubble sort starts by comparing the first two elements in the array. If element 0 is greater than element 1, they are swapped. The array would then appear as shown in Figure 8-2.

Figure 8-2 Elements 0 and 1 are swapped



This method is repeated with elements 1 and 2. If element 1 is greater than element 2, they are swapped. The array would then appear as shown in Figure 8-3.

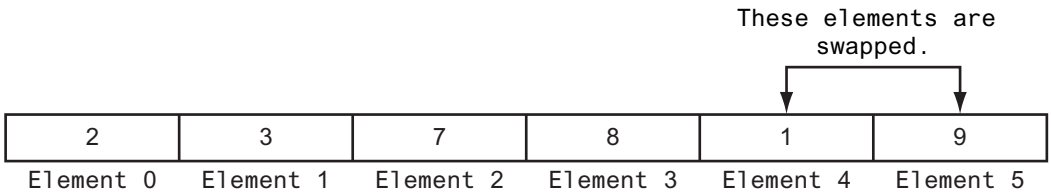
Figure 8-3 Elements 1 and 2 are swapped



Next, elements 2 and 3 are compared. In this array, these elements are already in the proper order (element 2 is less than element 3), so no values are swapped. As the cycle continues, elements 3 and 4 are compared. Once again, it is not necessary to swap the values because they are already in the proper order.

When elements 4 and 5 are compared, however, they must be swapped because element 4 is greater than element 5. The array now appears as shown in Figure 8-4.

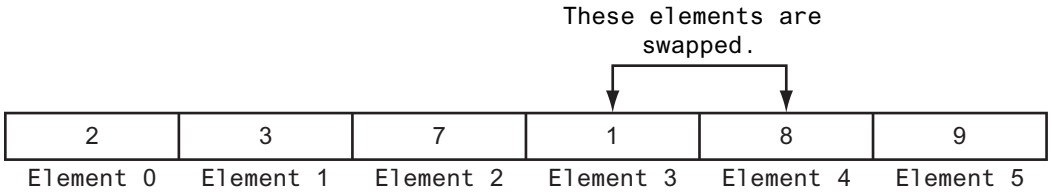
Figure 8-4 Elements 4 and 5 are swapped



At this point, the entire array has been scanned once, and the largest value, 9, is in the correct position. There are other elements, however, that are not yet in their final positions. So, the algorithm will make another pass through the array, comparing each element with its neighbor. In the next pass, it will stop comparing after reaching the next-to-last element because the last element already contains the correct value.

The second pass starts by comparing elements 0 and 1. Because those two are in the proper order, they are not swapped. Elements 1 and 2 are compared next, but once again, they are not swapped. This continues until elements 3 and 4 are compared. Because element 3 is greater than element 4, they are swapped. Element 4 is the last element that is compared during this pass, so this pass stops. The array now appears as shown in Figure 8-5.

Figure 8-5 Elements 3 and 4 are swapped



At the end of the second pass, the last two elements in the array contain the correct values. The third pass starts now, comparing each element with its neighbor. The third pass will not involve the last two elements, however, because they have already been sorted. When the third pass is finished, the last three elements will hold the correct values, as shown in Figure 8-6.

Figure 8-6 The array after the third pass



Each time the algorithm makes a pass through the array, the portion of the array that is scanned is decreased in size by one element, and the largest value in the scanned portion of the array is moved to its final position. When all of the passes have been made, the array will appear as shown in Figure 8-7.

Figure 8-7 The array with all elements sorted



Here is the bubble sort algorithm in pseudocode:

```
For maxElement = each subscript in the array, from the last to the first
   For index = 0 To maxElement - 1
      If array[index] > array[index + 1]
         swap array[index] with array[index + 1]
      End If
   End For
End For
```

And here is the C++ code for the bubble sort algorithm written as a function that sorts an array of integers. The function accepts two arguments: the array to sort, and the size of the array.

```
1  void bubbleSort(int array[], int size)
2  {
3     int maxElement;
4     int index;
5
6     for (maxElement = size - 1; maxElement > 0; maxElement--)
7     {
8        for (index = 0; index < maxElement; index++)
```

```
 9         {
10            if (array[index] > array[index + 1])
11            {
12               swap(array[index], array[index + 1]);
13            }
14         }
15      }
16   }
```

In lines 3 and 4, the following local variables are defined:

- The `maxElement` variable will hold the subscript of the last element that is to be compared to its immediate neighbor.
- The `index` variable is used as an array subscript in one of the loops.

The function uses two `for` loops, one nested inside another. The outer loop begins in line 6 as follows:

```
for (maxElement = size - 1; maxElement > 0; maxElement--)
```

This loop will iterate once for each element in the array. It causes the `maxElement` variable to take on all of the array's subscripts, from the highest subscript down to 0. After each iteration, `maxElement` is decremented by one.

The second loop, which is nested inside the first loop, begins in line 8 as follows:

```
for (index = 0; index < maxElement; index++)
```

This loop iterates once for each of the unsorted array elements. It starts `index` at 0 and increments it up through `maxElement` – 1. During each iteration, the comparison in line 10 is performed:

```
if (array[index] > array[index + 1])
```

This `if` statement compares the element at `array [index]` with its neighbor `array [index + 1]`. If the element's neighbor is greater, the `swap` function is called in line 12 to swap the two elements.

Swapping Array Elements

As you saw in the description of the bubble sort algorithm, certain elements are swapped as the algorithm steps through the array. Let's briefly discuss the process of swapping two items in computer memory. Assume we have the following variable declarations:

```
int a = 1;
int b = 9;
```

Suppose we want to swap the values in these variables so the variable `a` contains 9, and the variable `b` contains 1. At first, you might think that we only need to assign the variables to each other, like this:

```
// ERROR! The following does NOT swap the variables.
a = b;
b = a;
```

To understand why this doesn't work, let's step through the two lines of code. The first statement

```
a = b;
```

causes the value 9 to be assigned to a. But what happens to the value 1 that was previously stored in a? Remember, when you assign a new value to a variable, the new value replaces any value that was previously stored in the variable. So, the old value, 1, will be thrown away.

Then, the next statement is

```
b = a;
```

Because the variable a contains 9, this assigns 9 to b. After these statements execute, the variables a and b will both contain the value 9.

To successfully swap the contents of two variables, we need a third variable that can serve as a temporary storage location:

```
int temp;
```

Then, we can perform the following steps to swap the values in the variables a and b:

- Assign the value of a to temp.
- Assign the value of b to a.
- Assign the value of temp to b.

Here is the code for a swap function that swaps two int arguments:

```
void swap(int &a, int &b)
{
   int temp = a;
   a = b;
   b = temp;
}
```



NOTE: It is critical that we use reference parameters in the swap function, because the function must be able to change the values of the items that are passed to it as arguments.

Program 8-4 demonstrates the bubbleSort function in a complete program.

Program 8-4

```
1  // This program demonstrates the Bubble Sort algorithm.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototypes
6  void bubbleSort(int[], int);
7  void swap(int &, int &);
8
9  int main()
10 {
11    const int SIZE = 6;
12
13    // Array of unsorted values
14    int values[SIZE] = { 6, 1, 5, 2, 4, 3 };
15
```

```
16      // Display the unsorted array.
17      cout << "The unsorted values:\n";
18      for (auto element : values)
19        cout << element << " ";
20      cout << endl;
21
22      // Sort the array.
23      bubbleSort(values, SIZE);
24
25      // Display the sorted array.
26      cout << "The sorted values:\n";
27      for (auto element : values)
28         cout << element << " ";
29      cout << endl;
30
31      return 0;
32   }
33
34   //*************************************************************
35   // The bubbleSort function sorts an int array in ascending order. *
36   //*************************************************************
37   void bubbleSort(int array[], int size)
38   {
39      int maxElement;
40      int index;
41
42      for (maxElement = size - 1; maxElement > 0; maxElement--)
43      {
44         for (index = 0; index < maxElement; index++)
45         {
46            if (array[index] > array[index + 1])
47            {
48               swap(array[index], array[index + 1]);
49            }
50         }
51      }
52   }
53
54   //***************************************************
55   // The swap function swaps a and b in memory.      *
56   //***************************************************
57   void swap(int &a, int &b)
58   {
59      int temp = a;
60      a = b;
61      b = temp;
62   }
```

Program Output

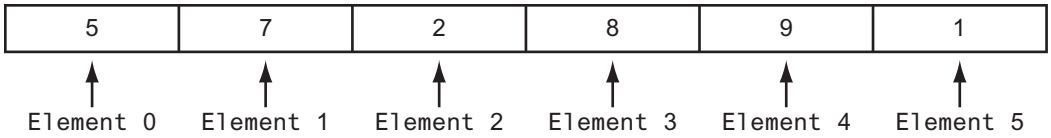
```
The unsorted values:
6 1 5 2 4 3
The sorted values:
1 2 3 4 5 6
```

VideoNote
The Selection Sort

The Selection Sort Algorithm

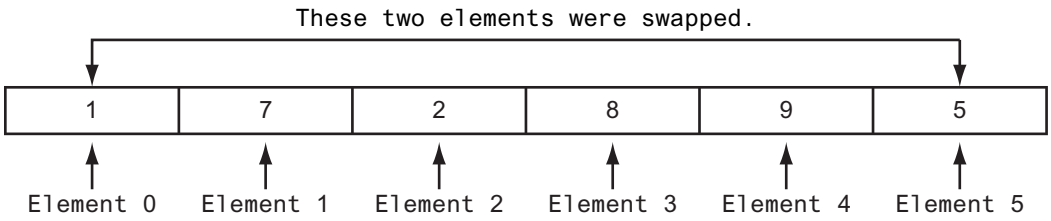
The bubble sort algorithm is simple, but it is inefficient because values move by only one element at a time toward their final destination in the array. The *selection sort algorithm* usually performs fewer swaps because it moves items immediately to their final position in the array. The selection sort works like this: The smallest value in the array is located and moved to element 0. Then, the next smallest value is located and moved to element 1. This process continues until all of the elements have been placed in their proper order. Let's see how the selection sort works when arranging the elements of the array in Figure 8-8.

Figure 8-8 Values in an array



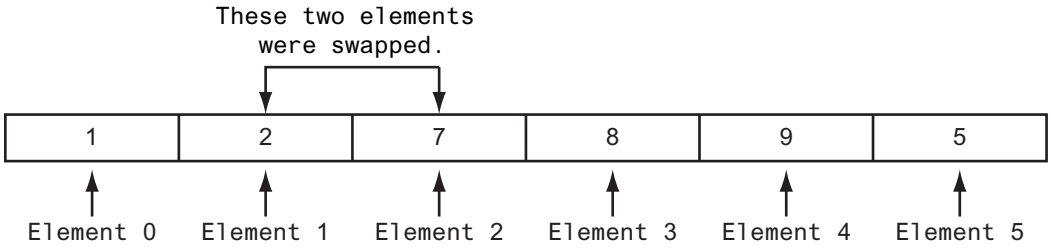
The selection sort scans the array, starting at element 0, and locates the element with the smallest value. Then, the contents of this element are swapped with the contents of element 0. In this example, the 1 stored in element 5 is swapped with the 5 stored in element 0. After the swap, the array appears as shown in Figure 8-9.

Figure 8-9 Values in the array after the first swap



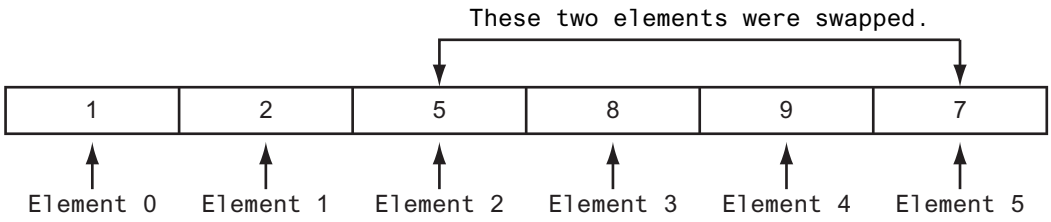
Then, the algorithm repeats the process, but because element 0 already contains the smallest value in the array, it can be left out of the procedure. This time, the algorithm begins the scan at element 1. In this example, the value in element 2 is swapped with the value in element 1. Then, the array appears as shown in Figure 8-10.

Figure 8-10 Values in the array after the second swap



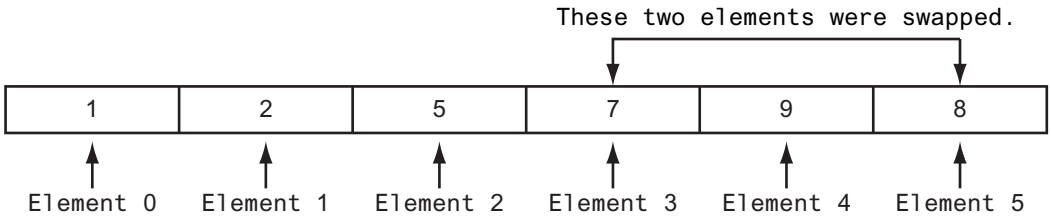
Once again the process is repeated, but this time the scan begins at element 2. The algorithm will find that element 5 contains the next smallest value. This element's value is swapped with that of element 2, causing the array to appear as shown in Figure 8-11.

Figure 8-11 Values in the array after the third swap



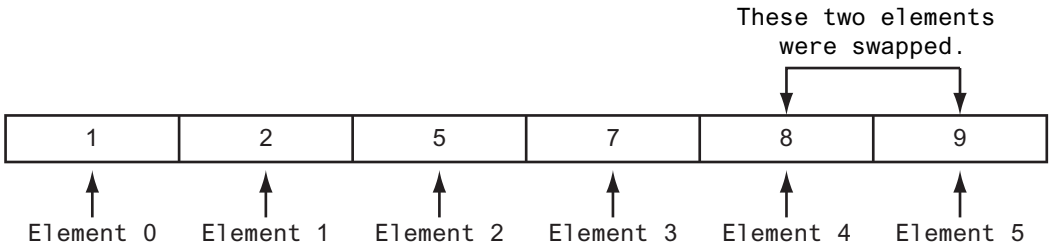
Next, the scanning begins at element 3. Its value is swapped with that of element 5, causing the array to appear as shown in Figure 8-12.

Figure 8-12 Values in the array after the fourth swap



At this point, there are only two elements left to sort. The algorithm finds that the value in element 5 is smaller than that of element 4, so the two are swapped. This puts the array in its final arrangement, as shown in Figure 8-13.

Figure 8-13 Values in the array after the fifth swap



Here is the selection sort algorithm in pseudocode:

```
For start = each array subscript, from the first to the next-to-last
   minIndex = start
   minValue = array[start]
   For index = start + 1 To size - 1
      If array[index] < minValue
         minValue = array[index]
```

```
            minIndex = index
         End If
      End For
      swap array[minIndex] with array[start]
   End For
```

The following C++ code implements the selection sort in a function. It accepts two arguments: an integer array, and the size of the array. The function uses the selection sort algorithm to arrange the values in the array in ascending order.

```
1  void selectionSort(int array[], int size)
2  {
3     int minIndex, minValue;
4
5     for (int start = 0; start < (size - 1); start++)
6     {
7        minIndex = start;
8        minValue = array[start];
9        for (int index = start + 1; index < size; index++)
10       {
11          if (array[index] < minValue)
12          {
13             minValue = array[index];
14             minIndex = index;
15          }
16       }
17       swap(array[minIndex], array[start]);
18    }
19 }
```

Inside the function are two `for` loops, one nested inside the other. The inner loop (in lines 9 through 16) sequences through the array, starting at `array[start + 1]`, searching for the element with the smallest value. When the element is found, its subscript is stored in the variable `minIndex`, and its value is stored in `minValue`. The outer loop (lines 5 through 18) then exchanges the contents of this element with `array[start]` and increments `start`. This procedure repeats until the contents of every element have been moved to their proper location.

Program 8-5 demonstrates the `selectionSort` function in a complete program.

Program 8-5

```
1  // This program demonstrates the Selection Sort algorithm.
2  #include <iostream>
3  using namespace std;
4
5  // Function prototypes
6  void selectionSort(int[], int);
7  void swap(int &, int &);
8
```

```
 9  int main()
10  {
11     const int SIZE = 6;
12
13     // Array of unsorted values
14     int values[SIZE] = { 6, 1, 5, 2, 4, 3 };
15
16     // Display the unsorted array.
17     cout << "The unsorted values:\n";
18     for (auto element : values)
19        cout << element << " ";
20     cout << endl;
21
22     // Sort the array.
23     selectionSort(values, SIZE);
24
25     // Display the sorted array.
26     cout << "The sorted values:\n";
27     for (auto element : values)
28        cout << element << " ";
29     cout << endl;
30
31     return 0;
32  }
33
34  //*************************************************************************
35  // The selectionSort function sorts an int array in ascending order. *
36  //*************************************************************************
37  void selectionSort(int array[], int size)
38  {
39     int minIndex, minValue;
40
41     for (int start = 0; start < (size - 1); start++)
42     {
43        minIndex = start;
44        minValue = array[start];
45        for (int index = start + 1; index < size; index++)
46        {
47           if (array[index] < minValue)
48           {
49              minValue = array[index];
50              minIndex = index;
51           }
52        }
53        swap(array[minIndex], array[start]);
54     }
55  }
56
57  //************************************************
58  // The swap function swaps a and b in memory.      *
59  //************************************************
60  void swap(int &a, int &b)
61  {
```

(program continues)

Program 8-5 *(continued)*

```
62      int temp = a;
63      a = b;
64      b = temp;
65   }
```

Program Output

```
The unsorted values:
6 1 5 2 4 3
The sorted values:
1 2 3 4 5 6
```

8.4 Focus on Problem Solving and Program Design: A Case Study

Like the previous case study, this is a program developed for the Demetris Leadership Center. Recall that DLC, Inc., publishes books, DVDs, and audio CDs. (See Table 8-1 for a complete list of products, with title, description, product number, and price.) Table 8-4 shows the number of units of each product sold during the past six months.

Table 8-4 Units Sold

Product Number	Units Sold
914	842
915	416
916	127
917	514
918	437
919	269
920	97
921	492
922	212

The vice president of sales has asked you to write a sales-reporting program that displays the following information:

- A list of the products in the order of their sales dollars (NOT units sold), from highest to lowest
- The total number of all units sold
- The total sales for the 6-month period

Variables

Table 8-5 lists the variables needed.

Table 8-5 Variables

Variable	Description
`NUM_PRODS`	A constant integer initialized with the number of products that DLC, Inc., sells. This value will be used in the definition of the program's array.
`prodNum`	Array of `int`s. Holds each product's number.
`units`	Array of `int`s. Holds each product's number of units sold.
`prices`	Array of `double`s. Holds each product's price.
`sales`	Array of `double`s. Holds the computed sales amounts (in dollars) of each product.

The elements of the four arrays, `prodNum`, `units`, `prices`, and `sales`, will correspond with each other. For example, the product whose number is stored in `prodNum[2]` will have sold the number of units stored in `units[2]`. The sales amount for the product will be stored in `sales[2]`.

Modules

The program will consist of the functions listed in Table 8-6.

Table 8-6 Functions

Function	Description
`main`	The program's `main` function. It calls the program's other functions.
`calcSales`	Calculates each product's sales.
`dualSort`	Sorts the `sales` array so the elements are ordered from highest to lowest. The `prodNum` array is ordered so the product numbers correspond with the correct sales figures in the sorted `sales` array.
`swap`	Swaps the values of two `double`s that are passed by reference (overloaded).
`swap`	Swaps the values of two `int`s that are passed by reference (overloaded).
`showOrder`	Displays a list of the product numbers and sales amounts from the sorted `sales` and `prodNum` arrays.
`showTotals`	Displays the total number of units sold and the total sales amount for the period.

Function `main`

Function `main` is very simple. It contains the variable definitions and calls the other functions. Here is the pseudocode for its executable statements:

```
Call calcSales
Call dualSort
Set display mode to fixed point with two decimal places of precision
Call showOrder
Call showTotals
```

Here is its actual C++ code:

```
// Calculate each product's sales.
calcSales(units, prices, sales, NUM_PRODS);
```

```
// Sort the elements in the sales array in descending
// order and shuffle the ID numbers in the id array to
// keep them in parallel.
dualSort(id, sales, NUM_PRODS);

// Set the numeric output formatting.
cout << setprecision(2) << fixed << showpoint;

// Display the products and sales amounts.
showOrder(sales, id, NUM_PRODS);

// Display total units sold and total sales.
showTotals(sales, units, NUM_PRODS);
```

The named constant `NUM_PRODS` will be defined globally and initialized to the value 9.

The arrays `id`, `units`, and `prices` will already be initialized with data. (It will be left as an exercise for you to modify this program so the user may enter these values.)

The `calcSales` Function

The `calcSales` function multiplies each product's units sold by its price. The resulting amount is stored in the `sales` array. Here is the function's pseudocode:

```
For index = each array subscript from 0 through the last subscript
   sales[index] = units[index] * prices[index]
End For
```

And here is the function's actual C++ code:

```
void calcSales(const int units[], const double prices[],
               double sales[], int num)
{
   for (int index = 0; index < num; index++)
     sales[index] = units[index] * prices[index];
}
```

The `dualSort` Function

The `dualSort` function is a modified version of the selection sort algorithm shown in Program 8-5. The `dualSort` function accepts two arrays as arguments: the `sales` array and the `id` array. The function actually performs the selection sort on the `sales` array. When the function moves an element in the `sales` array, however, it also moves the corresponding element in the `id` array. This is to ensure that the product numbers in the `id` array still have subscripts that match their sales figures in the `sales` array.

The `dualSort` function is also different in another way: It sorts the array in descending order. Here is the pseudocode for the `dualSort` function:

```
For start = each array subscript, from the first to the next-to-last
   index = start
   maxIndex = start
   tempId = id[start]
   maxValue = sales[start]
   For index = start + 1 To size - 1
```

```
            If sales[index] > maxValue
               maxValue = sales[index]
               tempId = id[index]
               maxIndex = index
            End If
         End For
         swap sales[maxIndex] with sales[start]
         swap id[maxIndex] with id[start]
      End For
```

Here is the actual C++ code for the `dualSort` function:

```
void dualSort(int id[], double sales[], int size)
{
   int start, maxIndex, tempid;
   double maxValue;

   for (start = 0; start < (size - 1); start++)
   {
      maxIndex = start;
      maxValue = sales[start];
      tempid = id[start];
      for (int index = start + 1; index < size; index++)
      {
         if (sales[index] > maxValue)
         {
            maxValue = sales[index];
            tempid = id[index];
            maxIndex = index;
         }
      }
      swap(sales[maxIndex], sales[start]);
      swap(id[maxIndex], id[start]);
   }
}
```



NOTE: Once the `dualSort` function is called, the `id` and `sales` arrays are no longer synchronized with the `units` and `prices` arrays. Because this program doesn't use `units` and `prices` together with `id` and `sales` after this point, it will not be noticed in the final output. However, it is never a good programming practice to sort parallel arrays in such a way that they are out of synchronization. It will be left as an exercise for you to modify the program so all the arrays are synchronized and used in the final output of the program.

The Overloaded swap Functions

We have two overloaded versions of the `swap` function in this program: one that swaps `double` values, and another that swaps `int` values. We need two different versions of the `swap` function because the `dualSort` function swaps two elements of the `sales` array, which are `doubles`, then it swaps two elements of the `id` array, which are `ints`. Here is the C++ code for both functions:

```
//*******************************************************
// The swap function swaps doubles a and b in memory. *
//*******************************************************
void swap(double &a, double &b)
{
   double temp = a;
   a = b;
   b = temp;
}

//*******************************************************
// The swap function swaps ints a and b in memory.     *
//*******************************************************
void swap(int &a, int &b)
{
   int temp = a;
   a = b;
   b = temp;
}
```

The showOrder Function

The showOrder function displays a heading and the sorted list of product numbers and their sales amounts. It accepts the id and sales arrays as arguments. Here is its pseudocode:

```
Display heading
For index = each subscript of the arrays from 0 through the last subscript
   Display id[index]
   Display sales[index]
End For
```

Here is the function's actual C++ code:

```
void showOrder(const double sales[], const int id[], int num)
{
   cout << "Product Number\tSales\n";

   cout << "---------------------------------\n";
   for (int index = 0; index < num; index++)
   {
       cout << id[index] << "\t\t$";
       cout << setw(8) << sales[index] << endl;
   }
   cout << endl;
}
```

The showTotals Function

The showTotals function displays the total number of units of all products sold and the total sales for the period. It accepts the units and sales arrays as arguments. Here is its pseudocode:

```
totalUnits = 0
totalSales = 0.0
```

```
For index = each array subscript from 0 through the last subscript
   Add units[index] to totalUnits[index]
   Add sales[index] to totalSales
End For
Display totalUnits with appropriate heading
Display totalSales with appropriate heading
```

Here is the function's actual C++ code:

```
void showTotals(const double sales[], const int units[], int num)
{
   int totalUnits = 0;
   double totalSales = 0.0;
   for (int index = 0; index < num; index++)
   {
       totalUnits += units[index];
       totalSales += sales[index];
   }
   cout << "Total Units Sold: " << totalUnits << endl;
   cout << "Total Sales:     $" << totalSales << endl;
}
```

The Entire Program

Program 8-6 shows the entire program's source code.

Program 8-6

```
1  // This program produces a sales report for DLC, Inc.
2  #include <iostream>
3  #include <iomanip>
4  using namespace std;
5
6  // Function prototypes
7  void calcSales(const int[], const double[], double[], int);
8  void showOrder(const double[], const int[], int);
9  void dualSort(int[], double[], int);
10 void showTotals(const double[], const int[], int);
11 void swap(double&, double&);
12 void swap(int&, int&);
13
14 int main()
15 {
16    // NUM_PRODS is the number of products produced.
17    const int NUM_PRODS = 9;
18
19    // Array with product ID numbers
20    int id[NUM_PRODS] = { 914, 915, 916, 917, 918,
21                          919, 920, 921, 922 };
22
23    // Array with number of units sold for each product
24    int units[NUM_PRODS] = { 842, 416, 127, 514, 437,
25                             269, 97,  492, 212 };
```

(program continues)

Program 8-6 *(continued)*

```
26
27      // Array with product prices
28      double prices[NUM_PRODS] = { 12.95, 14.95, 18.95, 16.95, 21.95,
29                                   31.95, 14.95, 14.95, 16.95 };
30
31      // Array to hold the computed sales amounts
32      double sales[NUM_PRODS];
33
34      // Calculate each product's sales.
35      calcSales(units, prices, sales, NUM_PRODS);
36
37      // Sort the elements in the sales array in descending
38      // order and shuffle the ID numbers in the id array to
39      // keep them in parallel.
40      dualSort(id, sales, NUM_PRODS);
41
42      // Set the numeric output formatting.
43      cout << setprecision(2) << fixed << showpoint;
44
45      // Display the products and sales amounts.
46      showOrder(sales, id, NUM_PRODS);
47
48      // Display total units sold and total sales.
49      showTotals(sales, units, NUM_PRODS);
50      return 0;
51  }
52
53  //*************************************************************
54  // Definition of calcSales. Accepts units, prices, and sales  *
55  // arrays as arguments. The size of these arrays is passed    *
56  // into the num parameter. This function calculates each      *
57  // product's sales by multiplying its units sold by each unit's *
58  // price. The result is stored in the sales array.            *
59  //*************************************************************
60
61  void calcSales(const int units[], const double prices[], double sales[], int num)
62  {
63     for (int index = 0; index < num; index++)
64        sales[index] = units[index] * prices[index];
65  }
66
67  //*************************************************************
68  // Definition of function dualSort. Accepts id and sales arrays *
69  // as arguments. The size of these arrays is passed into size.  *
70  // This function performs a descending order selection sort on  *
71  // the sales array. The elements of the id array are exchanged  *
72  // identically as those of the sales array. size is the number  *
73  // of elements in each array.                                   *
74  //*************************************************************
75
```

```
76  void dualSort(int id[], double sales[], int size)
77  {
78     int start, maxIndex, tempid;
79     double maxValue;
80
81     for (start = 0; start < (size - 1); start++)
82     {
83        maxIndex = start;
84        maxValue = sales[start];
85        tempid = id[start];
86        for (int index = start + 1; index < size; index++)
87        {
88           if (sales[index] > maxValue)
89           {
90              maxValue = sales[index];
91              tempid = id[index];
92              maxIndex = index;
93           }
94        }
95        swap(sales[maxIndex], sales[start]);
96        swap(id[maxIndex], id[start]);
97     }
98  }
99
100 //*********************************************************
101 // The swap function swaps doubles a and b in memory. *
102 //*********************************************************
103 void swap(double &a, double &b)
104 {
105    double temp = a;
106    a = b;
107    b = temp;
108 }
109
110 //*********************************************************
111 // The swap function swaps ints a and b in memory.    *
112 //*********************************************************
113 void swap(int &a, int &b)
114 {
115    int temp = a;
116    a = b;
117    b = temp;
118 }
119
120 //*****************************************************************
121 // Definition of showOrder function. Accepts sales and id arrays *
122 // as arguments. The size of these arrays is passed into num.    *
123 // The function first displays a heading, then the sorted list   *
124 // of product numbers and sales.                                 *
125 //*****************************************************************
126
```

(program continues)

Program 8-6 *(continued)*

```
127  void showOrder(const double sales[], const int id[], int num)
128  {
129     cout << "Product Number\tSales\n";
130     cout << "---------------------------------\n";
131     for (int index = 0; index < num; index++)
132     {
133        cout << id[index] << "\t\t$";
134        cout << setw(8) << sales[index] << endl;
135     }
136     cout << endl;
137  }
138
139  //*************************************************************
140  // Definition of showTotals function. Accepts sales and id arrays *
141  // as arguments. The size of these arrays is passed into num.     *
142  // The function first calculates the total units (of all          *
143  // products) sold and the total sales. It then displays these     *
144  // amounts.                                                       *
145  //*************************************************************
146
147  void showTotals(const double sales[], const int units[], int num)
148  {
149     int totalUnits = 0;
150     double totalSales = 0.0;
151
152     for (int index = 0; index < num; index++)
153     {
154        totalUnits += units[index];
155        totalSales += sales[index];
156     }
157     cout << "Total units Sold:   " << totalUnits << endl;
158     cout << "Total sales:       $" << totalSales << endl;
159  }
```

Program Output

```
Product Number              Sales
---------------------------------
914                      $10903.90
918                      $ 9592.15
917                      $ 8712.30
919                      $ 8594.55
921                      $ 7355.40
915                      $ 6219.20
922                      $ 3593.40
916                      $ 2406.65
920                      $ 1450.15

Total Units Sold:          3406
Total Sales:             $58827.70
```

8.5 Sorting and Searching vectors (Continued from Section 7.11)

CONCEPT: **The sorting and searching algorithms you have studied in this chapter may be applied to STL `vectors` as well as arrays.**

Once you have properly defined an STL `vector` and populated it with values, you may sort and search the `vector` with the algorithms presented in this chapter. Simply substitute the `vector` syntax for the array syntax when necessary. Program 8-7 demonstrates how to use the selection sort and binary search algorithms with a `vector` of strings.

Program 8-7

```
1  // This program demonstrates how to sort and search a
2  // string vector using selection sort and binary search.
3  #include <iostream>
4  #include <string>
5  #include <vector>
6  using namespace std;
7
8  // Function prototypes
9  void selectionSort(vector<string>&);
10 void swap(string &, string &);
11 int binarySearch(const vector<string>&, string);
12
13 int main()
14 {
15    string searchValue;  // Value to search for
16    int position;        // Position of found value
17
18    // Define a vector of strings.
19    vector<string> names{ "Lopez", "Smith", "Pike", "Jones",
20                          "Abernathy", "Hall", "Wilson", "Kimura",
21                          "Alvarado", "Harrison", "Geddes", "Irvine" };
22
23    // Sort the vector.
24    selectionSort(names);
25
26    // Display the vector's elements.
27    cout << "Here are the sorted names:\n";
28    for (auto element : names)
29       cout << element << endl;
30    cout << endl;
31
32    // Search for a name.
33    cout << "Enter a name to search for: ";
34    getline(cin, searchValue);
35    position = binarySearch(names, searchValue);
```

(program continues)

Program 8-7 *(continued)*

```
36
37      // Display the results.
38      if (position != -1)
39         cout << "That name is found at position " << position << endl;
40      else
41         cout << "That name is not found.\n";
42
43      return 0;
44   }
45
46   //************************************************************************
47   // The selectionSort function sorts a string vector in ascending order.*
48   //************************************************************************
49   void selectionSort(vector<string> &v)
50   {
51      int minIndex;
52      string minValue;
53
54      for (int start = 0; start < (v.size() - 1); start++)
55      {
56         minIndex = start;
57         minValue = v[start];
58         for (int index = start + 1; index < v.size(); index++)
59         {
60            if (v[index] < minValue)
61            {
62               minValue = v[index];
63               minIndex = index;
64            }
65         }
66         swap(v[minIndex], v[start]);
67      }
68   }
69
70   //*****************************************************
71   // The swap function swaps a and b in memory.       *
72   //*****************************************************
73   void swap(string &a, string &b)
74   {
75      string temp = a;
76      a = b;
77      b = temp;
78   }
79
80   //*********************************************************************
81   // The binarySearch function performs a binary search on a            *
82   // string vector. array, which has a maximum of size elements,        *
83   // is searched for the number stored in value. If the number is       *
84   // found, its vector subscript is returned. Otherwise, -1 is          *
85   // returned indicating the value was not in the vector.               *
86   //*********************************************************************
```

```
 87
 88  int binarySearch(const vector<string> &v, string str)
 89  {
 90     int first = 0,             // First vector element
 91         last = v.size() - 1,   // Last vector element
 92         middle,                // Mid point of search
 93         position = -1;         // Position of search value
 94     bool found = false;        // Flag
 95
 96     while (!found && first <= last)
 97     {
 98        middle = (first + last) / 2; // Calculate mid point
 99        if (v[middle] == str)        // If value is found at mid
100        {
101           found = true;
102           position = middle;
103        }
104        else if (v[middle] > str)    // If value is in lower half
105           last = middle - 1;
106        else
107           first = middle + 1;       // If value is in upper half
108     }
109     return position;
110  }
```

Program Output with Example Input Shown in Bold

```
Here are the sorted names:
Abernathy
Alvarado
Geddes
Hall
Harrison
Irvine
Jones
Kimura
Lopez
Pike
Smith
Wilson

Enter a name to search for: Lopez Enter
That name is found at position 8
```

Program Output with Example Input Shown in Bold

```
Here are the sorted names:
Abernathy
Alvarado
Geddes
Hall
```

(program output continues)

Program 8-7 *(continued)*

```
Harrison
Irvine
Jones
Kimura
Lopez
Pike
Smith
Wilson

Enter a name to search for: Carter Enter
That name is not found.
```

Review Questions and Exercises

Short Answer

1. Why is the linear search also called "sequential search"?
2. If a linear search function is searching for a value that is stored in the last element of a 10,000-element array, how many elements will the search code have to read to locate the value?
3. In an average case involving an array of N elements, how many times will a linear search function have to read the array to locate a specific value?
4. A binary search function is searching for a value that is stored in the middle element of an array. How many times will the function read an element in the array before finding the value?
5. What is the maximum number of comparisons that a binary search function will make when searching for a value in a 1,000-element array?
6. Why is the bubble sort inefficient for large arrays?
7. Why is the selection sort more efficient than the bubble sort on large arrays?

Fill-in-the-Blank

8. The _____ search algorithm steps sequentially through an array, comparing each item with the search value.
9. The _____ search algorithm repeatedly divides the portion of an array being searched in half.
10. The average number of comparisons required in the linear search of N elements is _____.
11. The average number of comparisons required in the binary search of 16 elements is _____.
12. If an array is sorted in _____ order, the values are stored from lowest to highest.
13. If an array is sorted in _____ order, the values are stored from highest to lowest.

True or False

14. T F The bubble sort algorithm is very efficient in sorting a large list of elements because of the inherent simplicity of the algorithm.
15. T F A sorting algorithm always rearranges elements of a list in an ascending order.
16. T F Though the number of comparisons for both bubble sort and selection sort are the same, the number of exchanges is lesser for selection sort.
17. T F The *maximum* number of comparisons performed by the linear search on an array of N elements is N/2 (assuming the search values are consistently found).
18. Complete the following table calculating the average and maximum number of comparisons the linear search will perform, and the maximum number of comparisons the binary search will perform.

Array Size →	50 Elements	500 Elements	10,000 Elements	100,000 Elements	10,000,000 Elements
Linear Search (Average Comparisons)					
Linear Search (Maximum Comparisons)					
Binary Search (Maximum Comparisons)					

Programming Challenges

1. **Charge Account Validation**

   Write a program that lets the user enter a charge account number. The program should determine if the number is valid by checking for it in the following list:

   5658845 4520125 7895122 8777541 8451277 1302850
   8080152 4562555 5552012 5050552 7825877 1250255
   1005231 6545231 3852085 7576651 7881200 4581002

   The list of numbers above should be initialized in a single-dimensional array. A simple linear search should be used to locate the number entered by the user. If the user enters a number that is in the array, the program should display a message saying the number is valid. If the user enters a number that is not in the array, the program should display a message indicating the number is invalid.

2. **Overlapping Arrays**

   An array overlaps another array if all elements of the latter array exist in the former array. They need not necessarily be in the same order. For example, [1,7,3,4,2] overlaps [1,2,3] because 1, 2, and 3 exist in [1,7,3,4,2]. To make the implementa-

tion easy, [1,7,3,4,2] overlaps [1,1,2,3] as well. We don't need to check whether 1 appears twice in the first array.

Write a program that lets the user enter two arrays and displays whether the first array overlaps the second.

3. **Overlapping Arrays Using Sort**

   Solve Programming Challenge 2 differently by sorting the larger array and performing a binary search in the sorted array.



VideoNote
Solving the Charge Account Validation Modification Problem

4. **Charge Account Validation Modification**

   Modify the program you wrote for Problem 1 (Charge Account Validation) so it performs a binary search to locate valid account numbers. Use the selection sort algorithm to sort the array before the binary search is performed.

5. **Rainfall Statistics Modification**

   Modify the Rainfall Statistics program you wrote for Programming Challenge 2 of Chapter 7 (Rainfall Statistics). The program should display a list of months, sorted in order of rainfall, from highest to lowest.

6. **String Selection Sort**

   Modify the `selectionSort` function presented in this chapter so it sorts an array of strings instead of an array of `int`s. Test the function with a driver program. Use Program 8-8 as a skeleton to complete.

Program 8-8

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    const int NUM_NAMES = 20;
    string names[NUM_NAMES] = {"Collins, Bill", "Smith, Bart", "Allen, Jim",
                               "Griffin, Jim", "Stamey, Marty", "Rose, Geri",
                               "Taylor, Terri", "Johnson, Jill",
                               "Allison, Jeff", "Looney, Joe", "Wolfe, Bill",
                               "James, Jean", "Weaver, Jim", "Pore, Bob",
                               "Rutherford, Greg", "Javens, Renee",
                               "Harrison, Rose", "Setzer, Cathy",
                               "Pike, Gordon", "Holland, Beth" };
    // Insert your code to complete this program

    return 0;
}
```

7. **Binary String Search**

   Modify the `binarySearch` function presented in this chapter so it searches an array of strings instead of an array of `int`s. Test the function with a driver program. Use

Program 8-8 as a skeleton to complete. (The array must be sorted before the binary search will work.)

8. **Linear Search Algorithm**

   Write a program that asks the user to input elements in an array of doubles of a maximum size of 50. It should call a function that should use the linear search algorithm to do the following:

   - Find a given value; display "Not Found" message if value is not found.
   - Find the number of occurrences of the value.
   - Display the location of each of the occurrences.

9. **Overlapping Arrays Benchmarks**

   In Programming Challenges 2 and 3, we wrote different solutions for the same problem. We do not know how exactly each solution performed. Write a program with two functions, one for each version of the Programming Challenge. Make sure that each function keeps a count of how many statements have been executed.

   Then let the user input two sequences and print out how many statements each version of the solution needed to execute.

10. **Insertion of Element in a Sorted Array**

    Write a program to declare an array of size 15 initialized with 10 integers. Include a function `show_array` that can display the contents of the array. The program should then call a function `sort_array` to sort the array, and again call the `show_array` function to display the sorted array. It then calls a function `insert_sorted` that inserts an element in the appropriate position in the sorted array. Finally, the program calls `show_array` again to display the array after insertion of the value.

11. **Using Files—String Selection Sort Modification**

    Modify the program you wrote for Programming Challenge 6 (String Selection Sort) so it reads in 20 strings from a file. The data can be found in the names.txt file.

12. **Televoting**

    In the student sample programs for this book, you will find text files named 12_artists.txt and 12_votes.txt. The first file contains the names of 10 artists. The second file contains votes sent in by viewers for their favorite artist. Each line of the second file represents one vote, and contains only a number in the range from 1 to 10 to indicate which artist got that vote.

    Write a program that reads both files—the names must be stored in an array and the votes in a vector—and displays the result. Write a function that accepts two arguments: a `vector` that contains the contents of the files and an array with scores that must be filled. The function should store the number of votes per artist in this array. Write a second function that sorts the scores in descending order and, at the same time, makes sure that the array with artists is sorted in a corresponding order. Finally, a third function should display the results in descending order mentioning both the score and the name of the artist.

This page intentionally left blank